



INSTITUTO POLITÉCNICO NACIONAL



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

SISTEMAS EN CHIP

Reporte de la práctica 19

“Interrupciones Externas”

Grupo: 6CM1

Integrantes

- **García Escamilla Bryan Alexis**
- **Meléndez Macedonio Rodrigo**

Profesor

Aguilar Sánchez Fernando

Fecha de entrega: 06 de marzo de 2025

INTRODUCCIÓN

Interrupciones Externas

Una interrupción externa es una señal enviada al procesador por un dispositivo de hardware externo (como un sensor, un botón o un transceptor de red) que indica que un evento requiere atención inmediata.

Cuando esto ocurre, el procesador:

1. **Suspende** la ejecución del programa principal.
2. **Guarda** el estado actual (el Contador de Programa y registros) en la Pila.
3. **Salta** a una dirección de memoria específica llamada Vector de Interrupción.
4. **Ejecuta** una función especial denominada ISR (Interrupt Service Routine).
5. **Retorna** al punto exacto donde se quedó en el programa principal.

Tipos de disparo

Las interrupciones externas pueden configurarse para activarse bajo distintas condiciones eléctricas en el pin de entrada:

- **Por Nivel (Level Triggered):** La interrupción se genera mientras el pin se mantenga en un estado lógico (alto o bajo). Si el evento dura mucho tiempo, la ISR podría ejecutarse repetidamente.
- **Por Flanco (Edge Triggered):** Es el más común. La interrupción solo ocurre en la transición.
 - *Flanco de Subida:* Al pasar de 0 a 1.
 - *Flanco de Bajada:* Al pasar de 1 a 0.
 - *Cualquier cambio lógico:* Se activa tanto al subir como al bajar la señal.

Jerarquía y Prioridad

En sistemas complejos, pueden ocurrir varias interrupciones al mismo tiempo. Para gestionar esto, los procesadores utilizan un Controlador de Interrupciones:

- **Prioridad Fija:** Cada pin de interrupción tiene un nivel de importancia predefinido. Si dos señales llegan a la vez, el procesador atiende primero la de mayor prioridad.
- **Interrupciones Enmascarables:** Son aquellas que el programador puede "desactivar" temporalmente mediante software.
- **Interrupciones No Enmascarables (NMI):** Reservadas para eventos catastróficos, como una falla de energía o un error crítico de hardware. El procesador no puede ignorarlas.

Latencia de interrupción

Se refiere al tiempo que transcurre desde que la señal eléctrica llega al pin hasta que se ejecuta la primera instrucción de la ISR. Una latencia baja es vital para aplicaciones como el despliegue de una bolsa de aire en un auto o el control de un motor de alta velocidad.

DESARROLLO

Circuito en hecho en Proteus

Para el desarrollo de la práctica 19, el circuito a armar es el siguiente:

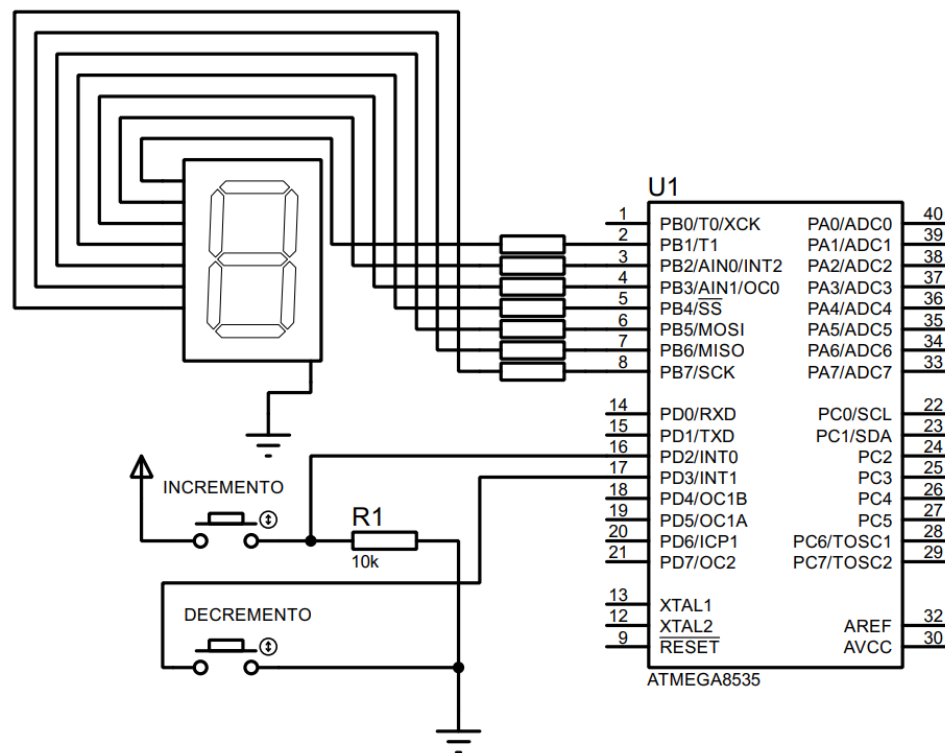


Imagen 1. Circuito creado en Proteus.

Configuración previa al código

De acuerdo con la imagen, el registro B se configura como de salida (PB1-PB7), mientras que el registro D como de entrada (PD2 y PD3), activando las resistencias de pull-up.

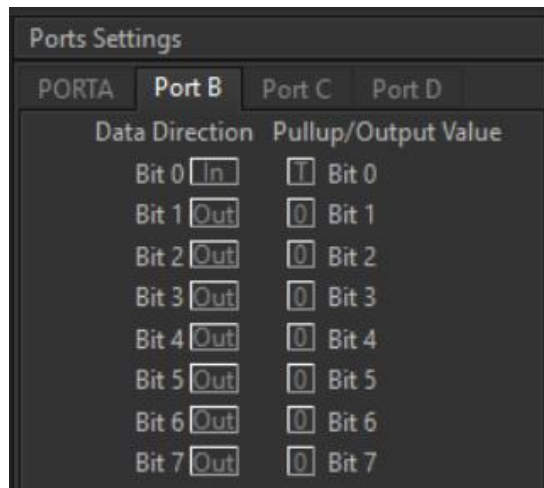


Imagen 2. Configuración del puerto B.

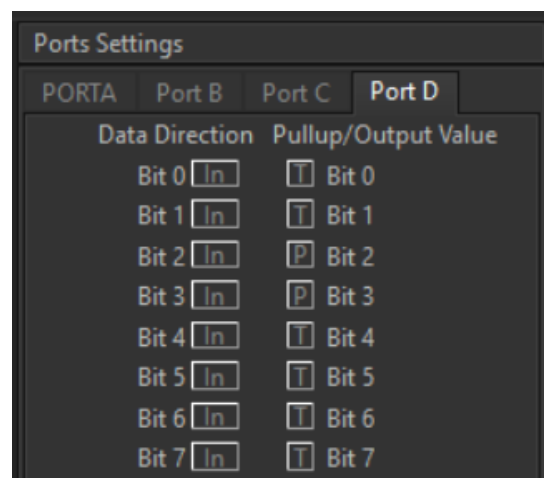


Imagen 3. Configuración del puerto D.

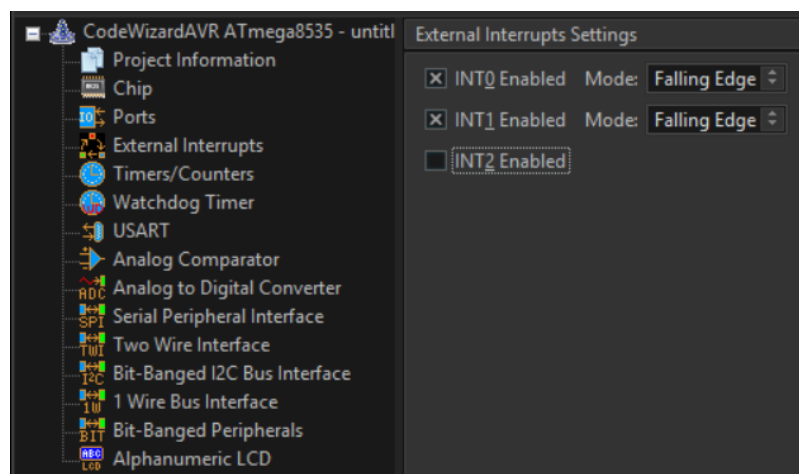


Imagen 4. Configuración de las interrupciones desde CodeVision.

C  digo del circuito de CodeVision:

```
1  /******  
2  This program was created by the CodeWizardAVR V4.06a  
3  Automatic Program Generator  
4      Copyright 1998-2025 Pavel Haiduc, HP InfoTech S.R.L.  
5  http://www.hpinfotech.ro  
6  
7  Project : Pr  ctica 19 'Interrupciones Externas'  
8  Version : 1.3  
9  Date    : 09/03/2026  
10 Author  : Garc  a Escamilla Bryan Alexis  
11 Company :  
12 Comments:  
13  
14  
15 Chip type      : ATmega8535  
16 Program type   : Application  
17 AVR Core Clock frequency: 1.000000 MHz  
18 Memory model   : Small  
19 External RAM size : 0  
20 Data Stack size : 128  
21 *****/  
22  
23 // I/O Registers definitions  
24 #include <mega8535.h>  
25 #include <delay.h>  
26  
27 // Declare your global variables here  
28 char indice = 0;
```

```
30 // External Interrupt 0 service routine  
31 interrupt [EXT_INT0] void ext_int0_isr(void) {  
32     // Place your code here  
33     indice++;  
34     if (indice == 16) indice = 0;  
35     delay_ms(50);  
36 }  
37  
38 // External Interrupt 1 service routine  
39 interrupt [EXT_INT1] void ext_int1_isr(void) {  
40     // Place your code here  
41     if (indice == 0) indice = 15;  
42     else indice--;  
43     delay_ms(50);  
44  
45 }
```

```

47 void main(void) {
48     // Declare your local variables here
49     const char hex[16] = {0x7e, 0x0c, 0xb6, 0x9e, 0xcc, 0xda, 0xfa, 0x0e, 0xfe, 0xde, 0xee, 0xf8, 0xbc, 0xf2, 0xe2};
50
51     // Input/Output Ports initialization
52     // Port A initialization
53     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
54     DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
55     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
56     PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);
57
58     // Port B initialization
59     // Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=In
60     DDRB=(1<<ddb7) | (1<<ddb6) | (1<<ddb5) | (1<<ddb4) | (1<<ddb3) | (1<<ddb2) | (1<<ddb1) | (0<<ddb0);
61     // State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=T
62     PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);
63
64     // Port C initialization
65     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
66     DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
67     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
68     PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);
69
70     // Port D initialization
71     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
72     DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
73     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=P Bit2=P Bit1=T Bit0=T
74     PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (1<<PORTD3) | (1<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);
75
76     // Timer/Counter 0 initialization
77     // Clock source: System Clock
78     // Clock value: Timer 0 Stopped
79     // Mode: Normal top=0xFF
80     // OC0 output: Disconnected
81     TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (0<<WGM01) | (0<<CS02) | (0<<CS01) | (0<<CS00);
82     TCNT0=0x00;
83     OCR0=0x00;
84
85     // Timer/Counter 1 initialization
86     // Clock source: System Clock
87     // Clock value: Timer1 Stopped
88     // Mode: Normal top=0xFFFF
89     // OC1A output: Disconnected
90     // OC1B output: Disconnected
91     // Noise Canceler: Off
92     // Input Capture on Falling Edge
93     // Timer1 Overflow Interrupt: Off
94     // Input Capture Interrupt: Off
95     // Compare A Match Interrupt: Off
96     // Compare B Match Interrupt: Off
97     TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);
98     TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) | (0<<CS11) | (0<<CS10);
99     TCNT1H=0x00;
100    TCNT1L=0x00;
101    ICR1H=0x00;
102    ICR1L=0x00;
103    OCR1AH=0x00;
104    OCR1AL=0x00;
105    OCR1BH=0x00;
106    OCR1BL=0x00;

```

```

108 // Timer/Counter 2 initialization
109 // Clock source: System Clock
110 // Clock value: Timer2 Stopped
111 // Mode: Normal top=0xFF
112 // OC2 output: Disconnected
113 ASSR=0<<AS2;
114 TCCR2=(0<<WGM20) | (0<<COM21) | (0<<COM20) | (0<<WGM21) | (0<<CS22) | (0<<CS21) | (0<<CS20);
115 TCNT2=0x00;
116 OCR2=0x00;
117
118 // Timer(s)/Counter(s) Interrupt(s) initialization
119 TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);
120
121 // External Interrupt(s) initialization
122 // INT0: On
123 // INT0 Mode: Falling Edge
124 // INT1: On
125 // INT1 Mode: Falling Edge
126 // INT2: Off
127 GICR=(1<<INT1) | (1<<INT0) | (0<<INT2);
128 MCUCR=(1<<ISC11) | (0<<ISC10) | (1<<ISC01) | (0<<ISC00);
129 MCUCSR=(0<<ISC2);
130 GIFR=(1<<INTF1) | (1<<INTF0) | (0<<INTF2);
131
132 // USART initialization
133 // USART disabled
134 UCSRB=(0<<RXCIE) | (0<<TXCIE) | (0<<UDRIE) | (0<<RXEN) | (0<<TXEN) | (0<<UCS22) | (0<<RXB8) | (0<<TXB8);

```

```

136 // Analog Comparator initialization
137 // Analog Comparator: Off
138 // The Analog Comparator's positive input is
139 // connected to the AIN0 pin
140 // The Analog Comparator's negative input is
141 // connected to the AIN1 pin
142 ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) | (0<<ACIS1) | (0<<ACIS0);
143 SFIOR=(0<<ACME);
144
145 // ADC initialization
146 // ADC disabled
147 ADCSRA=(0<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) | (0<<ADPS2) | (0<<ADPS1) | (0<<ADPS0);
148
149 // SPI initialization
150 // SPI disabled
151 SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) | (0<<SPR1) | (0<<SPR0);
152
153 // TWI initialization
154 // TWI disabled
155 TWCR=(0<<TWEA) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);
156
157 // Globally enable interrupts
158 #asm("sei")

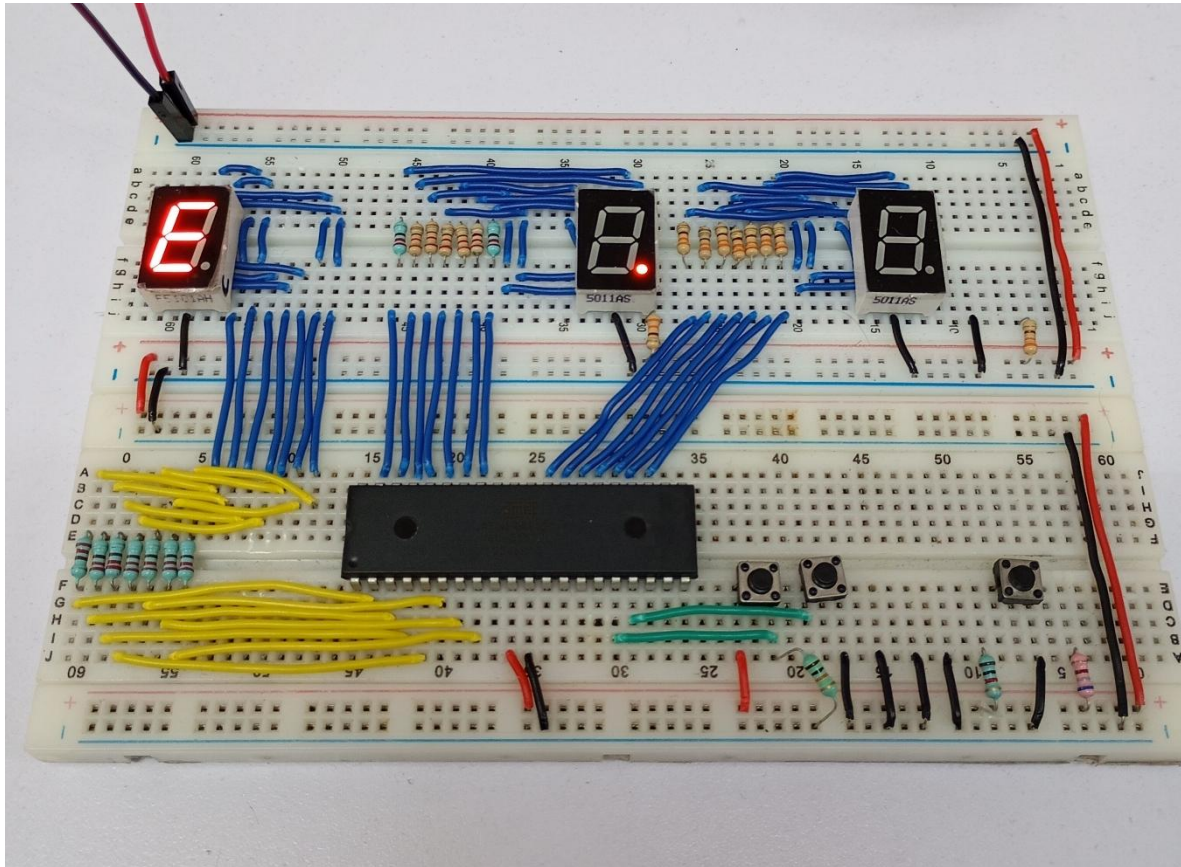
```

```

160 while (1) {
161     PORTB = hex[indice];
162     delay_ms(10);
163 }
164 }

```


Circuito físico



CONCLUSIÓN

- **García Escamilla Bryan Alexis**

Las interrupciones externas al igual que el convertidor analógico digital se pueden configurar desde CodeVision, para ello nos dirigimos a la sección 'External Interrupts' en la configuración del proyecto, esto nos mostrará las configuraciones de las interrupciones.

Se listan las tres interrupciones INT0, INT1 e INT2, pero en este caso solo se usaron las primeras dos, puesto que el ATMEGA tiene dos pines dedicados a ellas (PD2 y PD3). En esta sección se nos permite seleccionar el modo, es decir cuándo se va a ejecutar la interrupción, en este caso se eligió el modo 'Falling Edge' ya que hay recordar que los pines en donde están conectados los botones tienen activada la resistencia de pull-up, lo que da un uno lógico a la salida cuando el botón no está presionado y cero cuando se presiona, es decir este modo detecta el cambio de 1 a 0 para ejecutar la interrupción, que justamente es lo que sucede.

Por último, una parte importante es el retardo en las interrupciones. En este caso se coloca para evitar los rebotes al presionar el botón, sin embargo, no es recomendable puesto que puede bloquear otras interrupciones hacer lento el sistema, pero como en este caso es una práctica sencilla y corta, no tiene mucha repercusión en ese aspecto.

- **Meléndez Macedonio Rodrigo**

Para esta práctica desarrollamos un circuito en el que teníamos un contador de 0 a F controlado por interrupciones externas, las cuales facilitan mucho la cuestión del rebote pues en este caso es otra de las formas de eliminarlo.

Para implementar las interrupciones, CodeVisionAVR ofrece nuevamente las facilidades para implementar la función en el código del programa, por lo que una vez más la práctica resultó ser más sencillo de lo que esperábamos.

En primer lugar, desde la configuración del proyecto el software lista un total de 3 interrupciones en el que solo usamos las primeras dos dado que el ATMEGA solo permite esa cantidad, luego seleccionamos el modo en el que se va a ejecutar la interrupción; en nuestro caso pusimos "Falling Edge" para que detecte los flancos de bajada de 1 a 0. Y una vez configurado el proyecto de esa forma, lo creamos y únicamente el software se encargó de crear las funciones correspondientes que nos ayudaron en el desarrollo de esta práctica.

BIBLIOGRAFÍA

(s.f.). *Interrupciones Externas, Temporizadores y PWM*. Controles Digitales. Recuperado el 7 de marzo de 2026.
http://controlesdigitales.com/Libro_Felipe_Santiago/04_Cap_4_5_6_7.pdf